

Astro Website Development

The Grid and Charts Website is built using [Astro](#), a static site generator framework.

Within the website repositories, shared code is managed using [AG Website Shared](#) .

- [Gotchas](#)
- [Urls](#)
 - [Link to docs page without a framework](#)
- [Markdoc](#)
 - [Code snippets](#)
 - [Include partial markdoc files](#)
 - [Links](#)
 - [{% link %} tag](#)
 - [Conditional content](#)
 - [Custom Tags/Components](#)
 - [🌟 Creating Custom Tags/Components](#)
 - [Grid Example Runner \[Grid only\]](#)
 - [Chart Example Runner \[Charts only\]](#)
 - [API Reference \[Charts only\]](#)
 - [Tabs](#)
 - [Image Caption](#)
 - [Video](#)
 - [Note](#)
 - [Warning](#)
 - [Flex](#)
 - [Framework](#)
 - [Framework Capitalised \(Grid Only\)](#)
 - [Enterprise icon](#)
- [Environment variables](#)

Gotchas

Some things to note when working in Astro:

- When adding a React component to a `.astro` file, it will default to no client side JavaScript eg, `useEffect` code or using `localStorage`. To allow for `localStorage` use a [client directive](#). Use `client:load` for building the component at compile time and hydrating the client side code during run time or use `client:only` to load the component only during client side rendering

```
1 // Component.astro
2 <Component /> // No client side JS
3 <Component client:load /> // Build time + client side hydration
```

```
4 <Component client:only /> // Only client side hydration
```

• `src/contents` folder

- ⚠ Only use *lower case* to name content files. It will fail silently if capitals are used. See [collections ID is always lowercased · Issue #13265 · withastro/astro](#). The `collections` exported in `content.config.ts` can use upper case though eg, [ag-grid/documentation/ag-grid-docs/src/content.config.ts at 0345a38c1698f2864b9f991fab2b9dc407023d3a · ag-grid/ag-grid](#)
- When adding content and using `getEntry` to get the data, you may need to update `src/content.config.ts` schema for new data attributes or add validation
- Can't add `json` in `content/docs` folder, as the content needs to be the same type. Can add it with a `_` if need be, but probably best to add it to another `content` folder

Some things to note when working in Markdoc:

- HTML tags do not work in markdoc. Custom [tags](#) will need to be created for custom components
- Use double quotes (`"`) for attributes
- Can't add extra spaces to indent content (if we need it, we need to [update the markdoc config](#))
- Objects and arrays are a lot more strict eg,
 - ✅ `config={ key: 'value' }`
 - ❌ `config={ key : 'value' }` - space before `:`
 - ❌ `config= { key: 'value' }` - space after `=`
 - ❌ `config = { key: 'value' }` - space before `=`

Urls

There are few ways to get the url on website components:

Method	When to use it	Example
<code>urlWithBaseUrl</code>	Include the base url (<code>/</code> for grid, and <code>/charts</code> for charts). For root urls that don't require a framework	<code>urlWithBaseUrl('/community')</code> → charts

		/charts/community → grid /community
urlWithPrefix	For docs pages that require a framework prefix ./ framework page / root url like using urlWithBaseUrl (ignores framework)	urlWithPrefix({ framework: 'javascript', url: './doc-page' }) → charts /charts/javascript/ doc-page → grid /javascript- data-grid/doc-page
gridUrlWithPrefix	Grid docs page	gridUrlWithPrefix({ framework: 'javascript', url: './doc-page' }) → localhost https://localhost:4 610/javascript- data-grid/doc-page → staging https://grid- staging.ag- grid.com/javascript -data-grid/doc- page → archive/production https://www.ag- grid.com/javascript -data-grid/doc- page

⚠ This is only available on production once v33.3 is released (after 9th May 2025). It will work on staging before then

Sometimes it is useful to create a link to a docs page without knowing what framework to put in the link eg, blog posts, social media, root pages not on docs etc. NOTE: You *do not* need to do this on `.mdoc` pages, as links with `./` in front will point to the relevant framework page.

To do this, there are redirects set up to redirect to the framework a user has already selected on the website (or react by default).

- On Grid, add `/data-grid/[page-name]` for the redirect eg, <https://grid-staging.ag-grid.com/data-grid/getting-started/>
- On Charts, add `/r/[page-name]` for the redirect eg, <https://charts-staging.ag-grid.com/r/bar-series/>

Markdoc

[Markdoc](#) (`.mdoc`) is used to write markdown docs, and live in the [content](#) folder separated by their respective sections. Markdoc was chosen for it's focus on content (as opposed to interactive apps eg, MDX).

To see all available custom tags, see `markdoc.config.ts` > `tags`.

Code snippets

To add code snippets with syntax highlighting, use a triple tick ``` with the language after it

```
1 ```html
2 <p>Some text</p>
3 ```
```

To create code snippets that get transformed for the different frameworks use triple tick + language + `{% frameworkTransform=true %}`

```
1 ```js {% frameworkTransform=true %}
2 const gridOptions = {
3   ...
4 }
5 ```
```

You can also add extra attributes in the `{% .. %}` section

```
1 ```js {% frameworkTransform=true spaceBetweenProperties=true %}
2 const gridOptions = {
3   ...
```

```
4 }  
5 ...
```

⚠ If there are issues with prettier formatting, you can use the html version of [prettier-ignore](#).

eg, prettier will format angular html to go onto 1 line. To prevent this use `<!-- prettier-ignore -->`

```
1 <!-- prettier-ignore -->  
2 ```html  
3 <ag-charts  
4   [options]="options"  
5   class="chart"  
6   style="width: 400px; height: 400px; display: block">  
7 </ag-charts>  
8 ...
```

Include partial markdown files

Reference the partial file using relative paths

```
1 {% partial file="./[fileName].mdoc" /%}  
2  
3 eg, within a page  
4 {% partial file="./_component-interface-javascript.mdoc" /%}
```

Uses [native astro markdown implementation](#) of [markdown partials](#).

Links

To link to the same framework page use relative paths (`./`) eg, `[Axis Types]`
`(./axes-types/)`

To link to another framework page use absolute paths (`/`) eg,

```
1 {% if isFramework("react") %}  
2 ... See the [JavaScript](/javascript/installation/) documentation for more information.  
3 {% /if %}
```

❌ Do not use parent paths eg, `../`

`{% link %}` tag

For advanced uses cases for links, you can use the `{% link %}` tag.

For the Grid website, to add the grid version number in a path, use the `{% $agGridVersion %}` within the `href` attribute (it doesn't work with normal markdown links) eg,

```

1 {% link href="https://github.com/ag-grid/ag-grid/tree/v{% $agGridVersion %}/packages/ag-grid-
  community/dist" %}
2 GitHub Repository
3 {% /link %}

```

To open links in another tab eg, external links use `isExternal=true` in the `{% link %}` tag

```

1 {% link href="https://strftime.org/" isExternal=true %}strftime{% /link %}

```

Conditional content

To write content that is specific to a certain framework (`"javascript" | "react" | "angular" | "vue"`), you can use `isFramework("[framework]")` in an `if` tag eg,

```

1 {% if isFramework("javascript") %}
2 Content
3 {% /if %}

```

For content that is only for frameworks ie, not `javascript` , you can use `isNotJavascriptFramework()` eg,

```

1 {% if isNotJavascriptFramework() %}
2 Content
3 {% /if %}

```

Custom Tags/Components

To extend markdown, [custom markdoc tags](#) have been created, which are implemented in Astro or React components. Markdoc tags are configured in the [markdoc.config.ts](#) file.

🌟 Creating Custom Tags/Components

To create new components in markdoc, you need to create custom tags by following this process:

1. Decide if it is going to be shared between charts and grid or is just specific to one website?
 - [shared] Put your component in the `external/ag-website/shared/src/components` and share using [yarn run subrepo](#)
 - [specific to one site] Put your component into the respective `components` folder
 - If not sure, whether it will be shared or not, put it in the site `components` folder for the time being. It can always be migrated to the shared folder in future
2. Decide if the component is just a simple wrapper for html like content, or requires interactivity (ie, needs JavaScript to work)

- a. [simple component] If no interactivity is required, a *React* component can be created
 - b. [interactive component] If interactivity is required, you will need an *Astro* component. You can use the Astro component to wrap a React component if you need. NOTE: you cannot go the other way around ie, you can't include an Astro component within a React component.
3. Wire up the component by adding the component to `markdoc.config.ts` > `tags`. Follow the [markdoc structure for custom tags](#) for creating a new tag. You can add `attributes` to make the tag configurable.

⚠ The convention is to use *lower camel case* for the names of custom tags

- a. [simple component] For React depending on whether it is a `default` export or not:

- i. For default exported React component, add to the `render` path *without* the `.tsx` suffix:

```
1 // Default exported React component
2 const Idea: FunctionComponent<Props> = ({ children }) => { ... };
3 export default Idea;
4
5 // In custom tag config
6 import { component } from '@astrojs/markdoc/config';
7 import type { Render } from '@astrojs/markdoc/config';
8 import type { Config, Schema } from '@markdoc/markdoc';
9
10 export const idea: Schema<Config, Render> = {
11   render: component('../..external/ag-website-shared/src/components/alert/Idea'),
12 };
```

- ii. For named export React component, add to the `render` path *without* the `.tsx` suffix and add the name of the component as the 2nd argument to `render`:

```
1 // Named export React component
2 export const EnterpriseIcon: FunctionComponent = () => { ... }
3
4 // In custom tag config
5 import { component } from '@astrojs/markdoc/config';
6 import type { Render } from '@astrojs/markdoc/config';
7 import type { Config, Schema } from '@markdoc/markdoc';
8
9 export const enterpriseIcon: Schema<Config, Render> = {
10   render: component('../..external/ag-website-shared/src/components/icon/EnterpriseIcon', 'EnterpriseIcon'),
11 };
```

- b. [interactive component] For Astro, you need to include the `.astro` suffix eg,

- i.

```
1 import { component } from '@astrojs/markdoc/config';
2 import type { Render } from '@astrojs/markdoc/config';
3 import type { Config, Schema } from '@markdoc/markdoc';
4
```

```

5 export const image: Schema<Config, Render> = {
6   render: component('../..../external/ag-website-
  shared/src/components/image/Image.astro'),
7   // ...
8 }

```

c. [shared] You can also add this tag configuration into the [external/ag-website-shared/src/markdoc/tags](#) folder, so it's easier to share between websites

4. Use the new component in a markdoc (`.mdoc`) file (see below in this doc for example usages)

An example of a new shared component created:

 [Ag 13628 migrations overview by markdurrant · Pull Request #10051 · ag-grid/ag-grid](#) :

- [shared + interactive component]
 - `external/ag-website-shared/src/components/major-table/MajorTable.astro` - Astro component for interactivity
 - `external/ag-website-shared/src/markdoc/tags/majorTable.ts` - shared configuration for the custom tag
- `documentation/ag-grid-docs/markdoc.config.ts` - added custom tag to markdoc config file

Grid Example Runner [Grid only]

To show a grid example runner

```

1 {% gridExampleRunner
2   title="Context Menu Example"
3   name="context-menu"
4   typescriptOnly=false
5   suppressDarkMode=false
6   exampleHeight=900
7   consoleBufferSize=50
8 }/%}

```

Attributes

- `title`
- `name` - The name of the example within the `_examples` folder
- `typescriptOnly` - Only show the generated typescript example
- `suppressDarkMode` - Don't apply dark mode changes

- `exampleHeight` - Height of the example in pixels
- `consoleBufferSize` - Set the number of items to show in the console log component

In examples, to log to the developer console and show the logger component below the example, use `console.log` . ⚠ Do not use `window.alert` .

If the `main.ts` file contains any `console.log` s, an extra logging component will be automatically added to the bottom of the example. This is in addition to the developer console logs and shows logs of primitive types, and simple objects/arrays. These are the rules:

- All primitive types are logged as is, where primitive types are defined as `["string", "number", "boolean", "undefined", "null", "NaN", "symbol"]`
- Arrays where **all items** are primitive types are logged
 - Logged arrays are displayed on 1 line.
- Objects where **all values** are primitive types and there are **no more than 3** properties are logged
 - Logged objects are displayed formatted on multiple lines with 2 space indents
- All other values are logged as `[Object]` - see developer console

✓ Examples of console logs

```

1 console.log('string'); // string
2 console.log(23); // 23
3 console.log(null); // null
4 console.log(undefined); // undefined
5 console.log(NaN); // NaN
6 console.log(true, false); // true false
7 console.log('string', 23, null, undefined, true, false); // string 23 null undefined true
  false
8 console.log([]); // []
9 console.log(['string']); // ["string"]
10 console.log(['string', 23, null, undefined, true, false]); //
   ["string",23,null,null,true,false]
11 console.log([{' a: 'string', b: 23, c: null }, 23, null, undefined, true, false]); //
   [Object] - see developer console
12 console.log([{}]); // [Object] - see developer console
13
14 console.log({ a: 'string', b: 23, c: null });
15 // {
16 //   "a": "string",
17 //   "b": 23,
18 //   "c": null
19 // }
20 console.log({ a: undefined, b: NaN, c: null });
21 // {
22 //   "b": null,
```

```

23 // "c": null
24 // }
25
26 console.log({ a: 'more', b: 'here', c: 'now', d: 'too much' }); // [Object] - see
    developer console

```

Column Objects

Every column displayed in the grid is represented by a **Column Object** which has attributes, methods and events for interacting with the specific column e.g. `col.visible()`.

Columns can be accessed via Grid API methods, and provided as parameters from some **Grid Events**.

The **Column Reference** displays a list of functions available on the Column Object.

It is also possible to listen for **Column Events** by attaching an **Event Listener**.

Log All Columns Log All Column IDs

Make	Model	Price
Toyota	Celica	35000
Ford	Mondeo	32000
Porsche	Boxster	72000
BMW	MSD	60000
Aston Martin	DBX	190000

Console

[Object] - see developer console

["make", "model", "price"]

["make", "model", "price"]

Updating Columns

Columns can be controlled by updating the column state, or updating the column definition.

Column State should be used when restoring a users grid, for example saving and restoring column widths.

Update Column Definitions to modify properties that the user cannot control, and as such are not supported by Column State. Whilst column definitions can be used to change stateful properties, this can cause additional side effects.

Defining Columns
Referencing Columns
Column IDs
Column Objects
Updating Columns



Chart Example Runner [Charts only]

To show a chart example runner

```

1 {% chartExampleRunner
2   title="Data Updates"
3   name="data-updates"
4   type="generated"
5   options="{ \"enterprise\": true }"
6   /%}

```

Attributes

- title**
- name** - The name of the example within the `_examples` folder
- type** - Example type. Determines which code switcher options are available
- options** - Example runner options

API Reference [Charts only]

```

1 {% apiReference
2   id="AgChartInterface"
3   include=["download", "getImageDataURL"]
4   hideHeader=true
5   /%}

```

Attributes

- `id`
- `include` - interface names to include
- `exclude` - interface names to exclude
- `hideHeader`
- `displayFirst` - display names at the top

Tabs

Create tabs of content. Tab content can be anything including other markdown tags. ⚠ NOTE:

`id` is required because there could be multiple tabs on a page, and they could have the same label.

```
1 {% tabs %}
2
3 {% tabItem id="tab-1" label="Tab 1" %}
4 Tab content 1
5 {% /tabItem %}
6
7 {% tabItem id="tab-2" label="Tab 2" %}
8 Tab content 2
9 {% /tabItem %}
10
11 {% /tabs %}
```

eg,

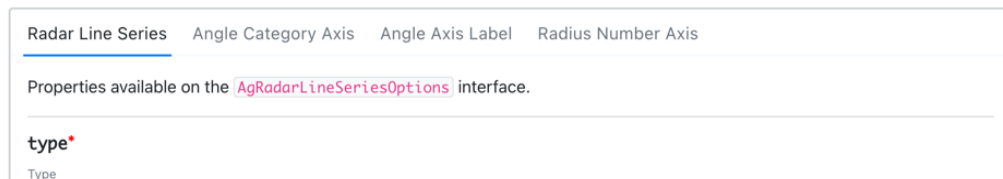


Image Caption

Show images with optional captions

```
1 {% imageCaption
2   pageName="charts-tooltips"
3   imageName="tooltip-no-title.png"
4   alt="Tooltip without the title element"
5   width="250px"
6   centered=true
7   constrained=true %}
8   No Title
9 {% /imageCaption %}
```

Attributes

- `pageName` - the folder name within `public/docs` where the image is stored
- `imageName` - name of image
- `alt` - alt value
- `centered` - center image
- `constrained` - restrict max width to 100%
- `descriptionTop` - place description on the top
- `width`
- `height`
- `minWidth`
- `maxWidth`
- `enableDarkModeFilter` - use CSS filter to create dark mode image eg, useful for svg files
- The caption text is the child text of the tag

Video

Add videos (preferred over gifs):

```
1 | {% video videoSrc="resources/annotations.mp4" centered=true %}
```

Attributes

- `videoSrc` - path to the video, relative to the markdoc page folder
- `centered` - centre the video on the page

The general process for creating screen recordings (on mac) is:

1. Use QuickTime Player to record your screen
2. Trim the video and save it as a `.mov` file
3. Copy the video into the markdoc page folder under `resources`
4. Run the `npm run compressVideo` script from the root folder. It will convert the `.mov` to `.mp4`, reduce the width (if necessary) and reduce the framerate to make the video smaller (the [settings are in these variables](#))

Note

Show notes

```
1 | {% note %}
```

```
2 Some note
3 {% /note %}
```

eg,



NOTE: We expect the options supplied to `AgChart.update()` to be the full configuration state to update to, not a partial configuration. Use `AgChart.updateDelta()` to apply partial updates.

Warning

Show warnings

```
1 {% warning %}
2 Some warning
3 {% /warning %}
```

eg,



Note that if you give the `bins` property you should not also give `binCount`, but if both are present `binCount` takes precedence.

Flex

Add a flex box around the content (NOTE: This should be used as a last resort, as there might be other solutions)

```
1 {% flex justifyContent="center" %}
2 {% someTag %}
3 {% someTag %}
4 {% /flex %}
```

Attributes

- `direction`
- `alignItems`
- `justifyContent`

Framework

Print the currently selected framework:

```
1 {% $framework %}
```

Framework Capitalised (Grid Only)

Print the currently selected framework with the correct capitalisation:

```
1 {% getFrameworkCapitalised() %}
```

Enterprise icon

Add inline enterprise icon

```
1 {% enterpriseIcon %}
```

eg, For text `Enterprise icon {% enterpriseIcon %}`

Enterprise icon 

Environment variables

In Astro, to use environment variables (values within `.env.*` files), use

`import.meta.env` (rather than `process.env`).

[Astro environment variables](#) are based off [Vite environment variables](#), which are statically replaced during build time. These are available in all server side code, but for client side code, you will need to add a `PUBLIC_` prefix to the variable before they will be exposed.

To use [default environment variables](#) on the client side, which do not have the `PUBLIC_` prefix eg, `import.meta.env.DEV`, pass them in on the top level astro layer eg,

```
1 ---
2 // page.astro
3 // Env var is passed in during build time
4 const isDev = import.meta.env.DEV
5 ---
6
7 <SomeComponent isDev={isDev} />
```

Example of error during build time, if you try to use `import.meta.env.DEV` (it will run fine on the dev server)

```
1 Unexpected token (Note that you need plugins to import files that are not JavaScript)
2 file: /Users/ttt/Work/Dev/ag-charts/packages/ag-charts-website/src/utils/env.ts:1:42
3 1: export const getIsDev = () => import.meta.env?.DEV;
```